МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ “ЛЬВІВСЬКА ПОЛІТЕХНІКА”

Кафедра систем штучного інтелекту



Лабораторна робота № 1

з дисципліни «Обробка зображень методами штучного інтелекту»

Виконав:

Студент групи КН-415

Лозовий Олег

Викладач:

Пелешко Д.Д.

Львів – 2025 р.

Тема: Попередня обробка зображень

Мета роботи: вивчити просторову фільтрацію зображень, методи мінімізації шуму, морфології, виділення країв і границь та елементи бібліотеки OpenCV для розв'язування цих завдань.

Варіант 12

Основне завдання (обов’язкове)

Вибрати з інтернету два зображення з різною деталізацією об’єктів та два зображення з різним контрастом. Без використання жодних бібліотек для обробки зображень (наприклад, OpenCV) виконати індивідуальне завдання та провести порівняльний аналіз отриманих результатів.

12. Виконати детекцію границь на зображеннях за допомогою операторів Roberts, Sobel.

Оригінальне зображення supra.jpg:



Детекція границь, за допомогою оператора Робертса по осі OX:



Детекція границь, за допомогою оператора Робертса по осі OY:



Детекція границь, за допомогою оператора Робертса по осі OX та OY одночасно:



Детекція границь, за допомогою оператора Собеля по осі OX:



Детекція границь, за допомогою оператора Собеля по осі OY:



Детекція границь, за допомогою оператора Собеля по осі OX та OY одночасно:



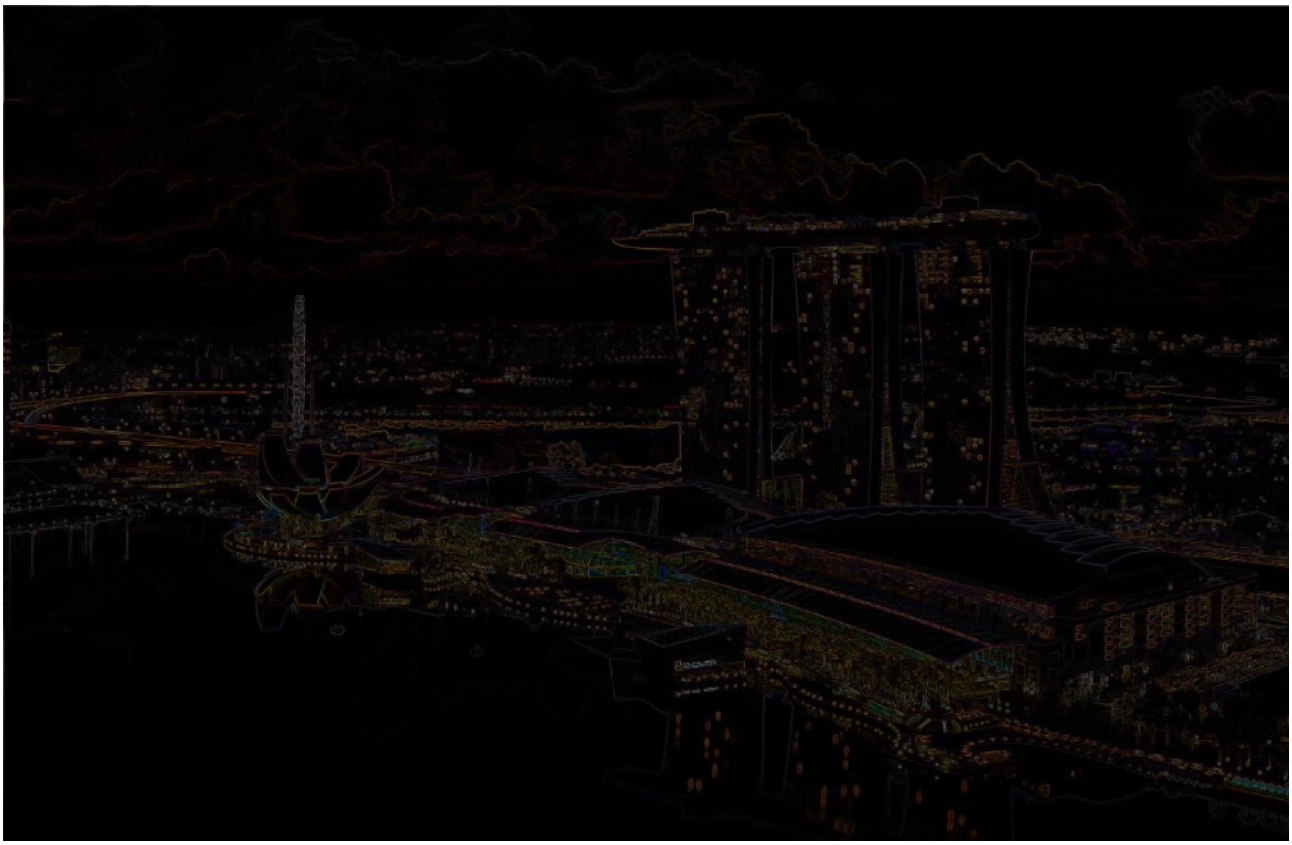
Оригінальне зображення tokio.jpeg:



Детекція границь, за допомогою оператора Робертса:



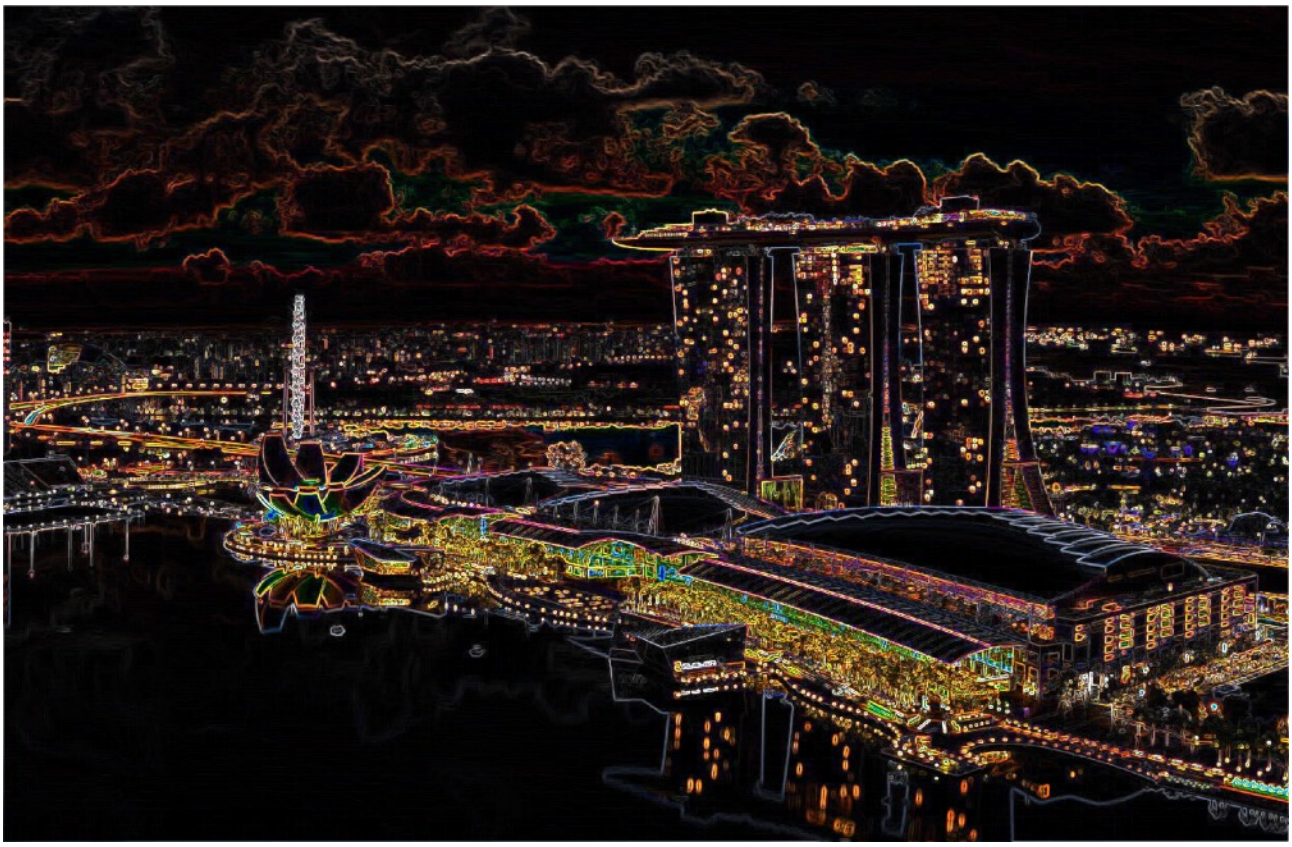
Детекція границь, за допомогою оператора Робертса у кольорі:



Детекція границь, за допомогою оператора Собеля:



Детекція границь, за допомогою оператора Собеля у кольорі:



Оригінальне зображення odissey.jpeg:



Детекція границь, за допомогою оператора Робертса:



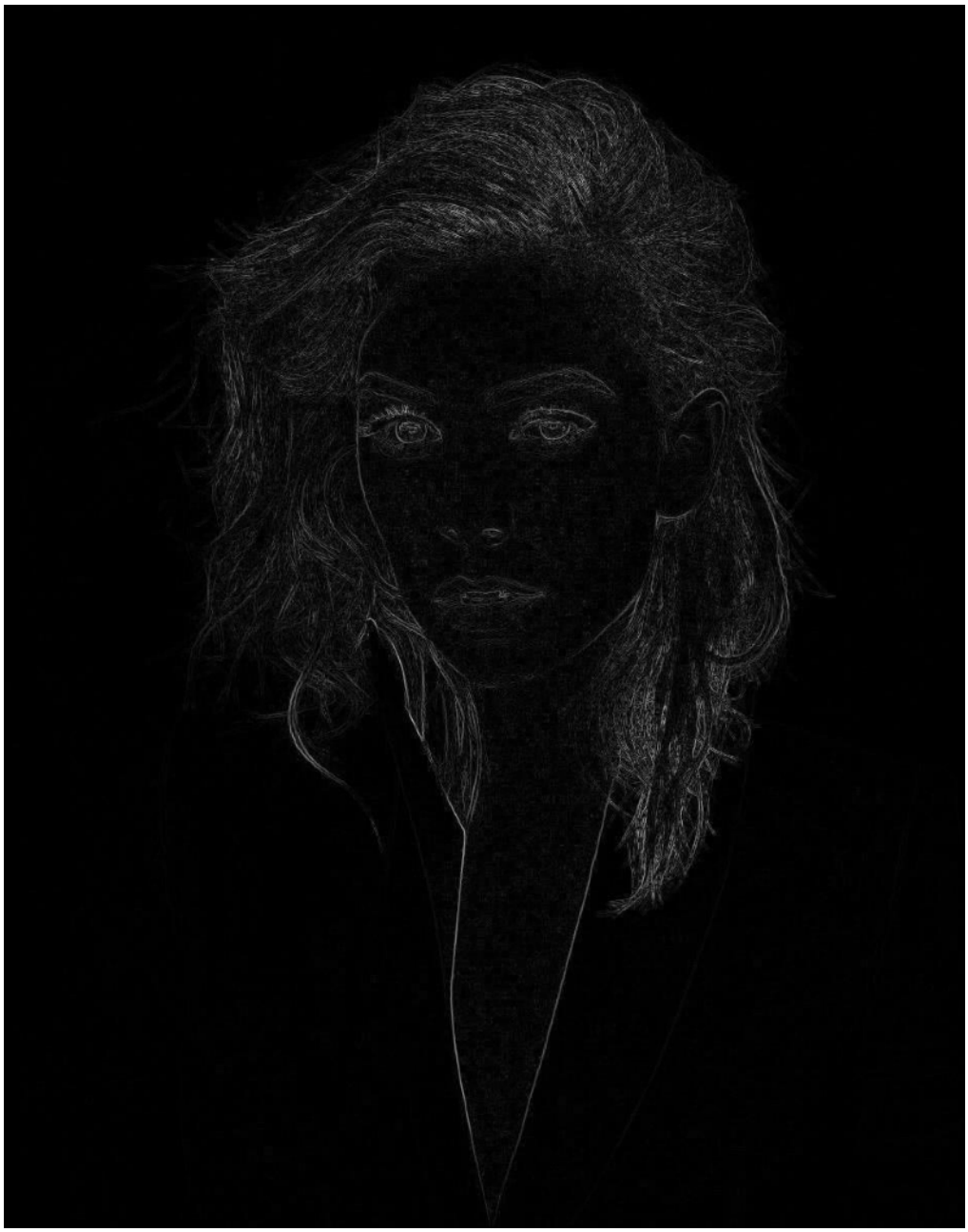
Детекція границь, за допомогою оператора Собеля:



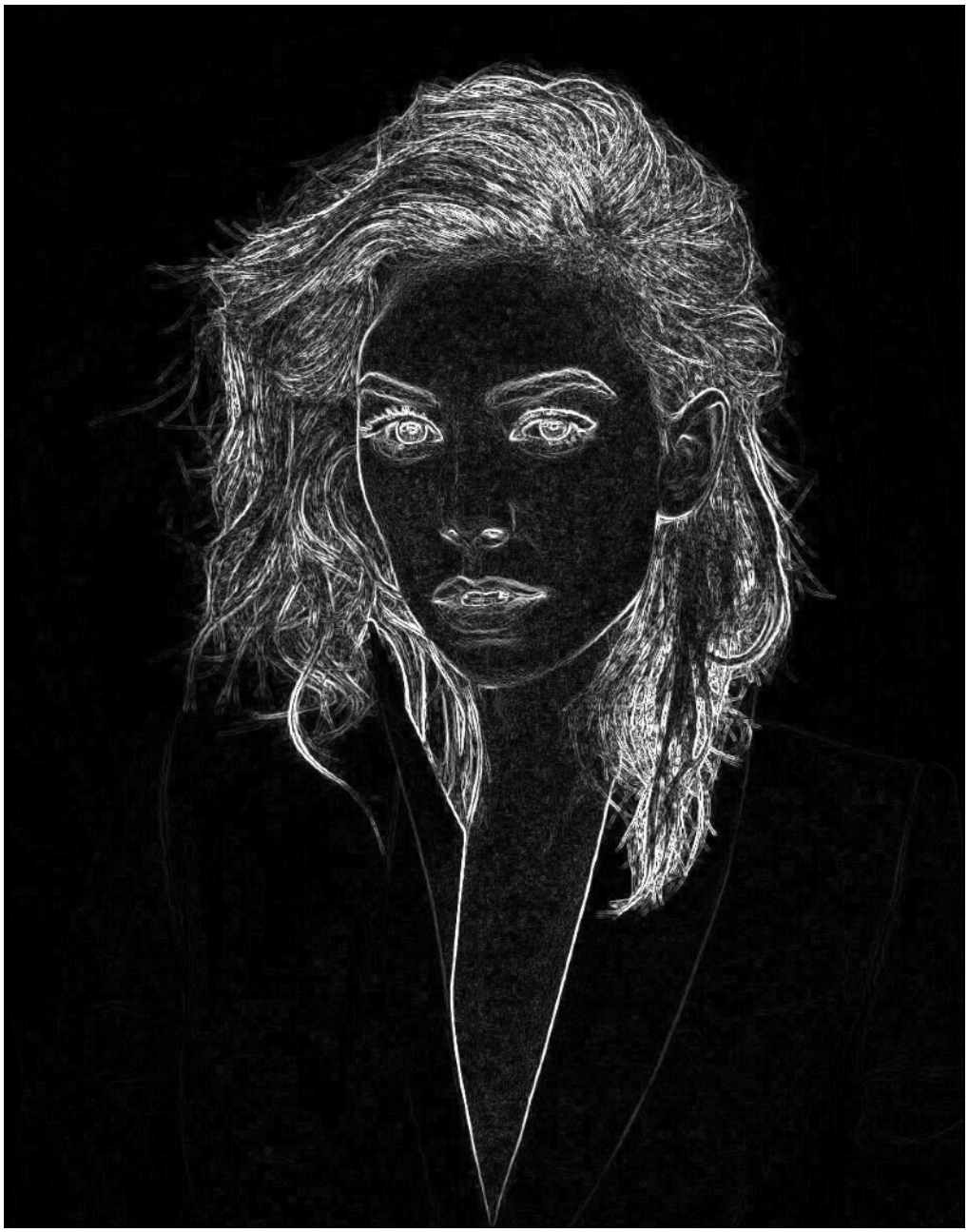
Оригінальне зображення venessa.jpg:



Детекція границь, за допомогою оператора Робертса:



Детекція границь, за допомогою оператора Собеля:



Додаткове завдання (за бажанням)

Вибрати один із методів морфолологічної обробки і написати приклад його застосування без використання сторонніх бібліотек.

Дилатація — це морфологічна операція, яка розширює об’єкти на зображенні, додаючи пікселі до їх країв. Це дозволяє з’єднувати розірвані частини об'єктів і заповнювати невеликі прогалини між ними. В результаті

деталі зображення стають більшими, що може допомогти в поліпшенні сегментації або відновленні контурів об’єктів.

Оригінальне зображення cars.jpeg:



Дилатація з kernel_size = 3:



Дилатація з kernel_size = 10:



Дилатація з kernel_size = 20:



Оригінальне зображення lp.jpg:



Дилатація з kernel_size = 4:



Дилатація з kernel_size = 8:



Дилатація з kernel_size = 16:



Висновок:

У ході виконання лабораторної роботи було досліджено методи попередньої обробки зображень, зокрема детекцію границь за допомогою операторів Робертса та Собеля, а також морфологічну операцію дилатації. Робота виконувалась без використання сторонніх бібліотек для обробки зображень, що дозволило глибше зрозуміти алгоритми їх реалізації.

Було проаналізовано результати детекції границь для різних зображень із різним рівнем деталізації та контрастності. Оператор Робертса та оператор Собеля продемонстрували свою ефективність у знаходженні границь на зображеннях. В той час як оператор Собеля з його маленьким ядром, краще справляється з зашумленими зображеннями, оператор Робертса краще справився на зображеннях з високим контрастом.

Крім того, було реалізовано операцію дилатації, яка дозволяє розширювати об'єкти на зображенні, заповнювати проміжки та з'єднувати розірвані області. Проведене тестування на різних значеннях kernel_size показало, що збільшення розміру ядра веде до сильнішого розширення контурів об'єктів.

ДОДАТОК

```
import numpy as np

from PIL import Image


def load_image(path):

    """ Завантажує зображення в grayscale"""

    image = Image.open(path).convert("L")

    return np.array(image, dtype=np.float32)


def roberts_operator(image):

    """ Оператор Робертса """

    Gx = np.array([[1, 0], [0, -1]], dtype=np.float32)

    Gy = np.array([[0, -1], [1, 0]], dtype=np.float32)

    edges = np.zeros_like(image)

    for i in range(image.shape[0] - 1):

        for j in range(image.shape[1] - 1):

            gx = np.sum(Gx * image[i:i + 2, j:j + 2])

            gy = np.sum(Gy * image[i:i + 2, j:j + 2])

            edges[i, j] = np.sqrt(gx ** 2 + gy ** 2)

    return edges



def sobel_operator(image):

    """ Оператор Собеля """

    Gx = np.array([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]],
dtype=np.float32)
```

```
    Gy = np.array([[1, 2, 1], [0, 0, 0], [-1, -2, -1]],
dtype=np.float32)
    edges = np.zeros_like(image)
    for i in range(1, image.shape[0] - 1):
        for j in range(1, image.shape[1] - 1):
            gx = np.sum(Gx * image[i - 1:i + 2, j - 1:j + 2])
            gy = np.sum(Gy * image[i - 1:i + 2, j - 1:j + 2])
            edges[i, j] = np.sqrt(gx ** 2 + gy ** 2)
    return edges



def dilate(image, kernel_size=4):
    """ Дилатація """
    kernel = np.ones((kernel_size, kernel_size), dtype=np.uint8)
    pad = kernel_size // 2
    padded_image = np.pad(image, ((pad, pad), (pad, pad), (0, 0)),
mode='constant', constant_values=0)
    output = np.zeros_like(image)
    for i in range(image.shape[0]):
        for j in range(image.shape[1]):
            for c in range(image.shape[2]):
                region = padded_image[i:i + kernel_size, j:j +
kernel_size, c]
                output[i, j, c] = np.max(region * kernel)
    return output
```

```
def load_color_image(path):
    """ Завантажує зображення в кольорі """
    image = Image.open(path).convert("RGB")
    return np.array(image, dtype=np.uint8)


def save_image(array, filename):
    image = Image.fromarray(np.clip(array, 0,
255).astype(np.uint8))
    image.save(filename)


def save_color_image(array, filename):
    image = Image.fromarray(array)
    image.save(filename)


image_path = "img.jpg"
image = load_image("original_photos/" + image_path)
roberts_edges = roberts_operator(image)
sobel_edges = sobel_operator(image)
save_image(roberts_edges, image_path + "roberts_edges.jpg")
save_image(sobel_edges, image_path + "sobel_edges.jpg")
img = load_color_image("original_photos/" + image_path)
dilate_edges = dilate(img)
save_color_image(dilate_edges, "test_img/" + image_path +
"dilate_edges.jpg")
```